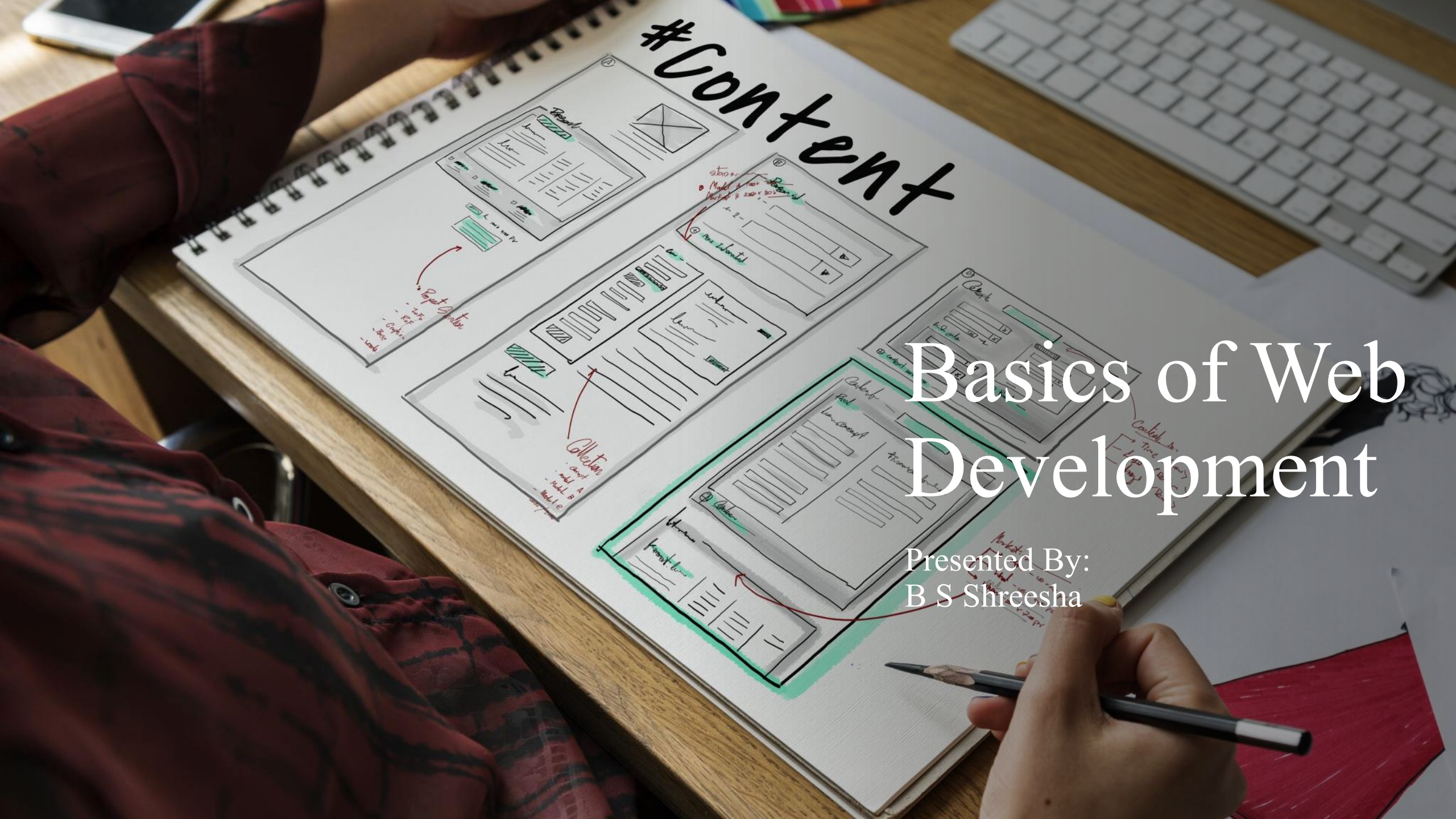


Basics of Web Development

Presented By:
B S Shreesha

Scan the QR Code for GitHub Repo Access

Inception of Internet

What is the Internet?

A global network that connects millions of private, public, academic, business, and government networks.

How Did It Start?

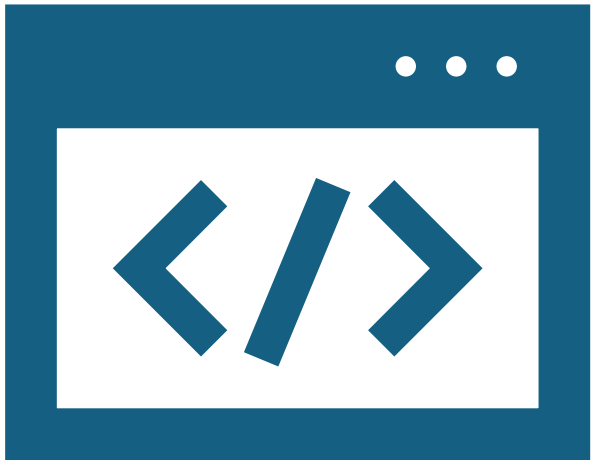
- **1960s:** The idea of connecting computers began during the Cold War.
- **ARPANET (1969):** The U.S. Department of Defense funded ARPANET, the world's first operational packet-switching network.
  - Connected four universities in the USA.
  - Allowed researchers to share data and communicate electronically.

Evolution of Internet

Key Milestones

- **1970s:** Introduction of email, the first “killer app” for the Internet.
- **1980s:** TCP/IP protocol developed—allowed different networks to connect and communicate.
- **1990:** Tim Berners-Lee invented the World Wide Web, making the Internet accessible to everyone.

What is Web Development?



Web development is the process of building and maintaining websites and web applications that run on the internet or an intranet. It includes everything from creating simple static web pages to complex dynamic web applications, social network services, and online business platforms.

Key areas include

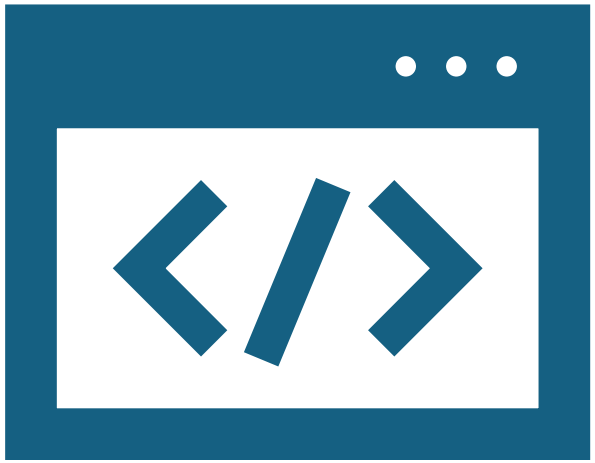
- **Frontend Development:** Deals with everything users see and interact with directly in their web browser.
  - ❖ Technologies: HTML, CSS, JavaScript, and frameworks/libraries like React, Angular, or Vue.js.

- **Backend Development:** Handles the "behind-the-scenes" logic and database interactions.
  - ❖ Technologies: Languages like Python, PHP, Java, Node.js, Ruby, and databases like MySQL, MongoDB.

- **Full Stack Development:** Combines both frontend and backend skills.

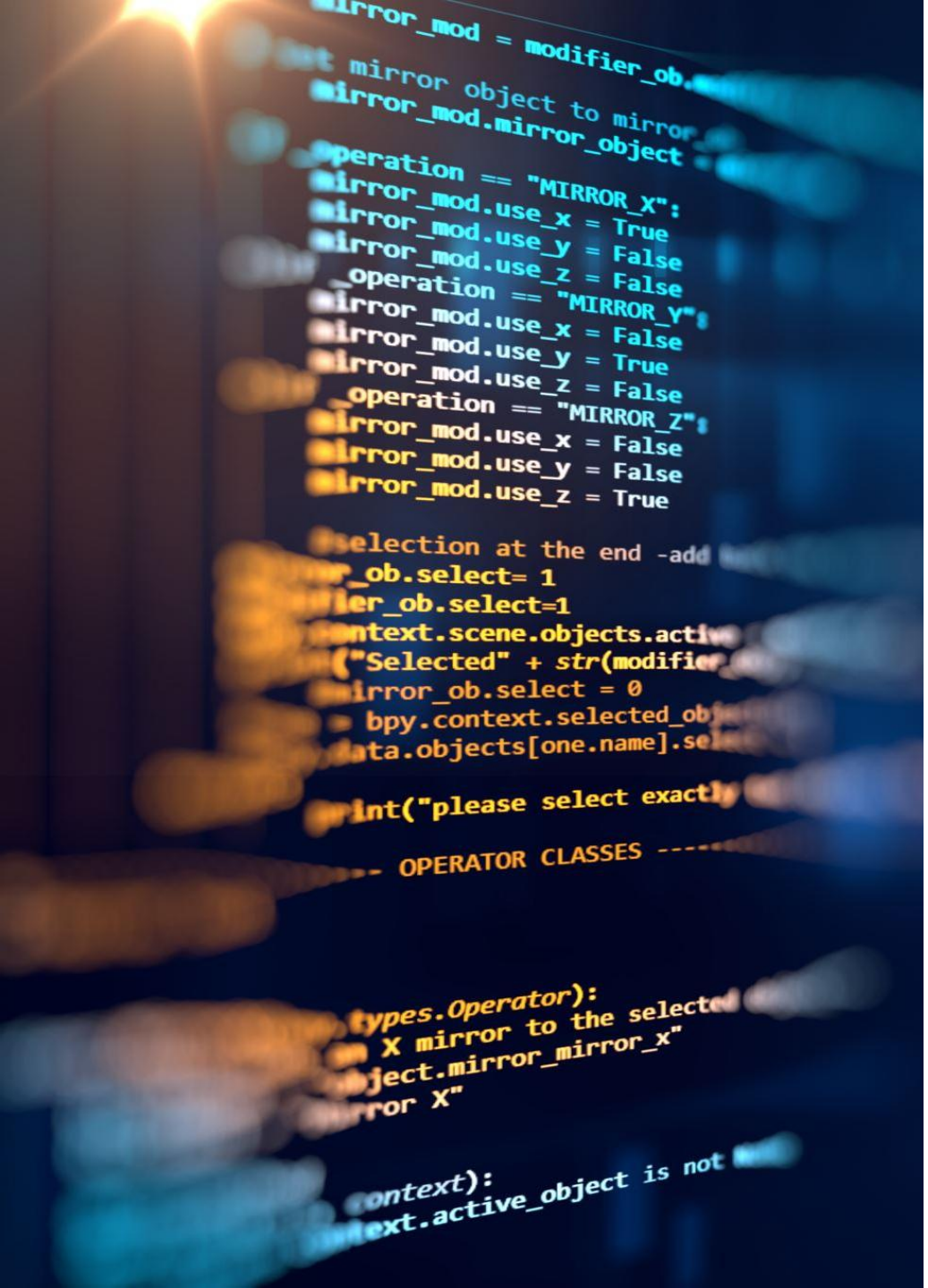
Other Aspects of Web Development

- **Web Design:** Focuses more on aesthetics and user experience (UI/UX).
- **Web Content:** Generation and management of content (text, images, videos).
- **Web Security:** Ensuring the website and user data are safe from threats.



Why is Web Development Important?

- The internet is integral to communication, business, education, and entertainment.
- Web development skills are in high demand as nearly every organization needs an online presence.



Introduction to HTML

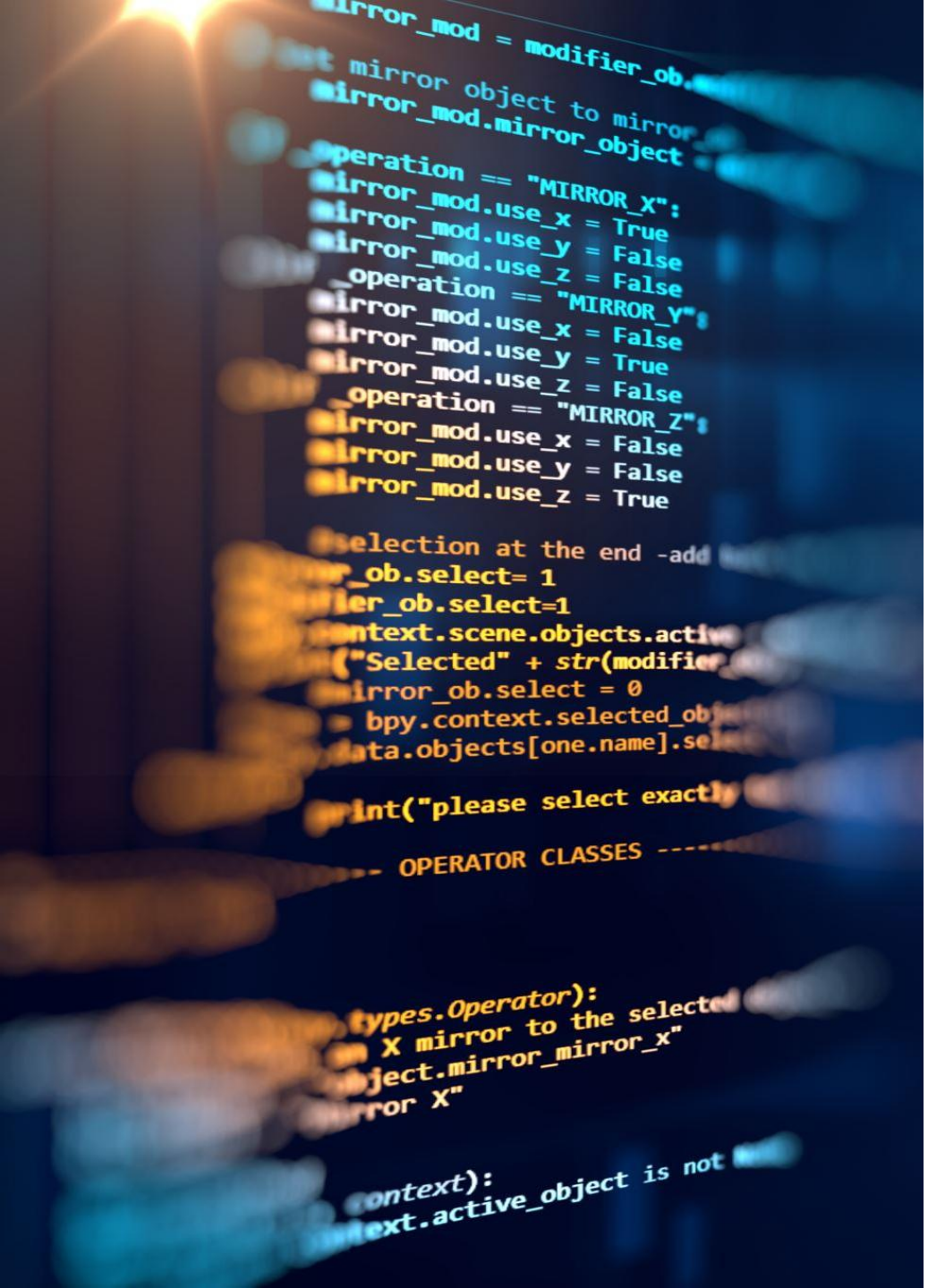
HTML stands for **HyperText Markup Language**. It is the standard language used to create and structure content on the web.

What is a Markup Language?

- A markup language uses tags to “mark up” sections of text to tell browsers how to display them.
- HTML is not a programming language; it does not have logic (like loops or conditions). It only structures content.

Origins: SGML

- **SGML (Standard Generalized Markup Language):** Developed in the 1980s as a standard for defining generalized markup languages for documents.
  - It is very flexible and powerful but complex for everyday use.
  - **HTML is a simplified version of SGML, designed for the web.**
  - SGML is the parent language; HTML inherits its concept of tags and document structure from SGML.



Introduction to HTML

HTML's Role in Web Development:

- HTML forms the **skeleton** of every webpage.
- It defines elements like headings, paragraphs, links, images, lists, tables, forms, etc.
- Browsers read HTML files and render them into visual web pages.

HTML Syntax:

HTML consists of elements wrapped in angle brackets (tags), e.g., <h1>, ,

Most elements have opening and closing tags: Web

Some elements are self-closing: <img src="image.jpg" alt="desc">

Basics of HTML - Syntax

Every HTML web page follows a standard structure to ensure browsers interpret and display the content correctly. Here's a breakdown:

1. <!DOCTYPE html>

- Declares the document type and HTML version.
- Tells the browser to render the page using the latest HTML standard (HTML5).
- Must be the very first line in any HTML document.

2. <html>

- Root element that wraps the entire HTML document.
- All other elements are nested inside <html>

Basics of HTML – Head Tag

3. <head>

- Contains meta-information about the page (not displayed to users).
- Common <head> elements:
  - ➢<title>: Sets the browser tab's title.
  - ➢<meta>: Provides metadata (character set, author, viewport for mobile, etc.).
  - ➢<link>: Links to external resources like CSS files.
  - ➢<style>: Internal CSS styles.
  - ➢<script>: JavaScript to execute before page loads.

```
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <title>Title</title>
5    </head>
6    <body>
7      Content goes here.
8    </body>
9  </html>
```

Basics of HTML – Body Tag

4. <body>

- Contains all the visible content of the web page.
- Elements such as headings, paragraphs, images, links, tables, and forms go here.

Why Is Structure Important?

- Ensures your page is correctly interpreted by browsers and search engines.
- Improves maintainability and scalability.
- Helps with accessibility (screen readers, etc.).

Headings and Paragraph

Headings (<h1> to <h6>)

- Used to define titles and subtitles on the page.
- <h1> is the most important heading (largest), <h6> is the least important (smallest).

```html
<p>This is a paragraph of text that explains something on your webpage.</p>
```

Paragraph

Used for regular blocks of text.

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <h1>Main Title</h1>
    <h2>Section Title</h2>
    <h3>Subsection Title</h3>
</body>
</html>
```

Why Use Headings and Paragraphs?

- They organize content for readers and search engines.
- Headings improve accessibility and navigation.

Links and Images

Links (<a>)

- Used to navigate between pages or to external websites.
- href attribute specifies the destination URL.

Visit Example.com

Images (<img>)

- Used to display pictures.
- **src** attribute specifies the image file location.
- **alt** attribute provides alternative text (important for accessibility).

<img src="logo.png" alt="Website Logo"> (Self closing)

Lists

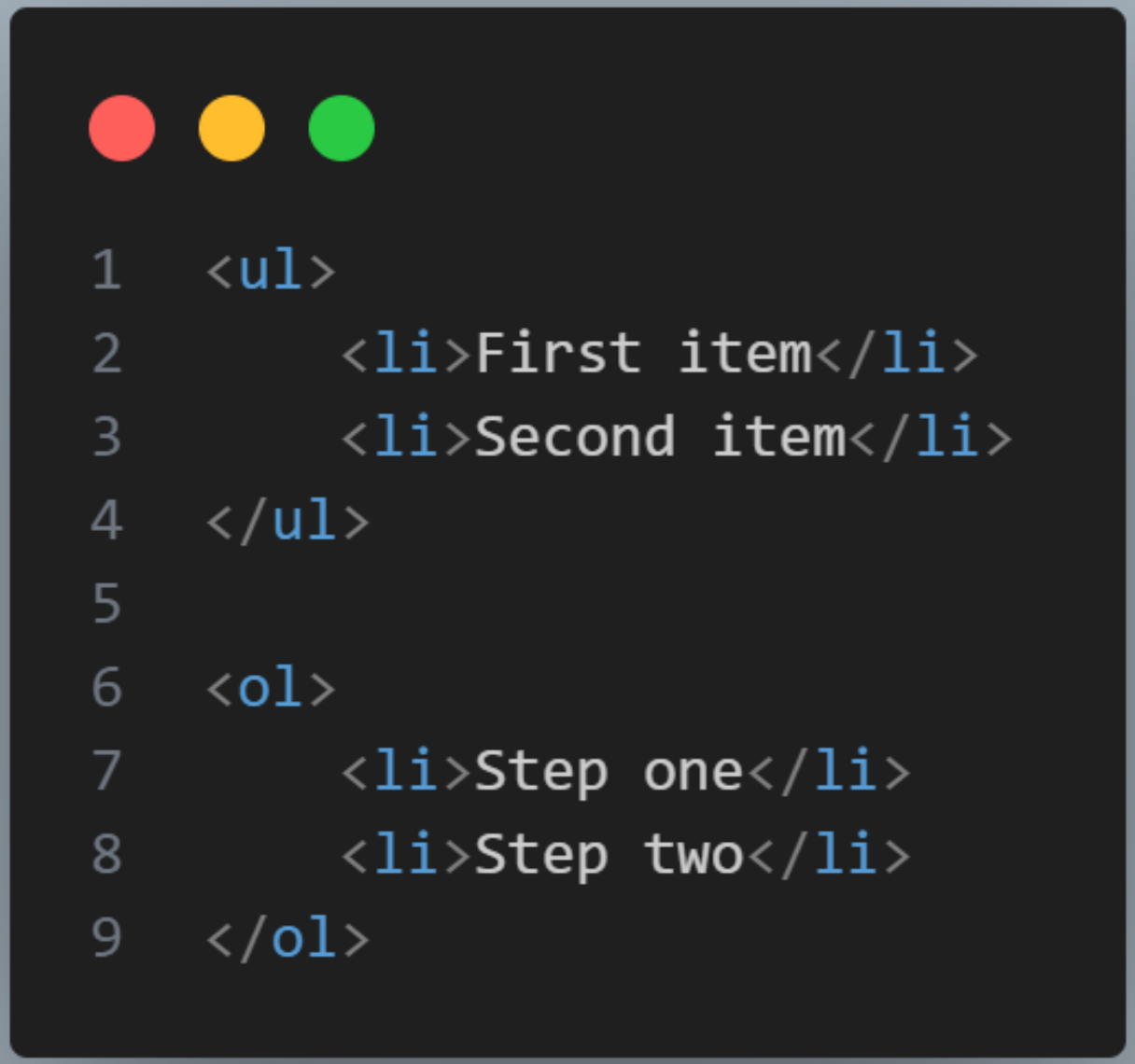
Lists

- Used to organize content in a sequence.

There are 2 kinds of list

a. Un-Ordered List: Begins with <ul> content present within

b. Ordered List: Begins with <ol> content present within



```
<ul>
    <li>First item</li>
    <li>Second item</li>
</ul>

<ol>
    <li>Step one</li>
    <li>Step two</li>
</ol>
```

Tables

Table

- Used to display data in rows and columns.

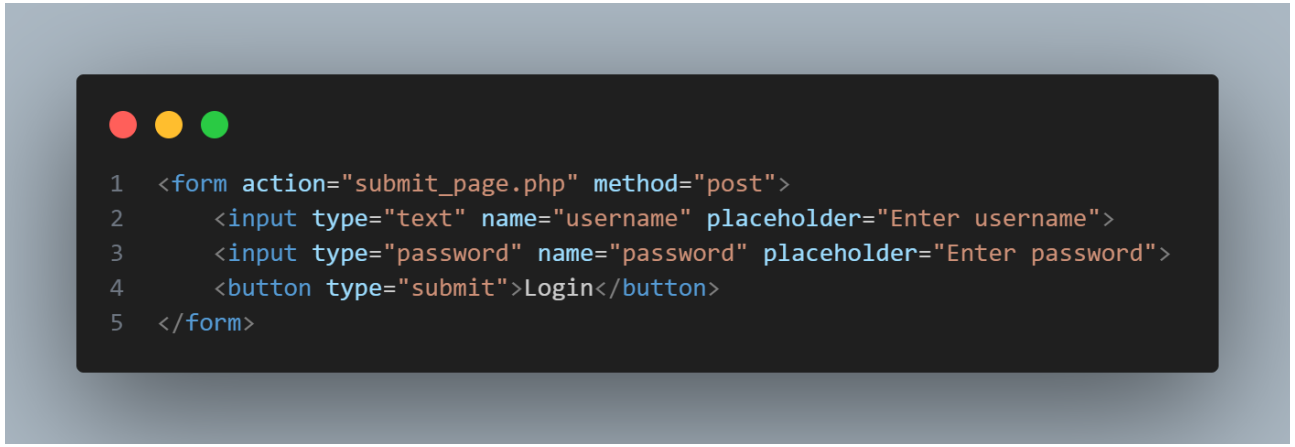
<tr>: Table row

<th>: Table heading

<td>: Table data

```html
<table>
    <tr>
        <th>Name</th>
        <th>Age</th>
    </tr>
    <tr>
        <td>Alice</td>
        <td>20</td>
    </tr>
    <tr>
        <td>Bob</td>
        <td>22</td>
    </tr>
</table>
```

Forms



```html
<form action="submit_page.php" method="post">
    <input type="text" name="username" placeholder="Enter username">
    <input type="password" name="password" placeholder="Enter password">
    <button type="submit">Login</button>
</form>
```

Forms are essential for collecting user input on websites—think of login pages, search boxes, or contact forms.

1. <form>

Wraps all input elements.

Attributes:

- **action**: URL where the form data is sent after submission.
- **method**: HTTP method (GET or POST)

2. Input Elements: Various types

3. Labels and Accessibility:

- Use <label> to describe inputs, improving accessibility.

4. Submitting the Form:

When the user clicks the submit button, the browser collects all input values and sends them to the server as specified by **action** and **method**

Input Methods

Input Type	Description	Example
text	Single-line text input	<input type='text'>
password	Password field (masked)	<input type='password'>
email	Email input	<input type='email'>
number	Numeric input	<input type='number'>
tel	Telephone number	<input type='tel'>
url	Website URL	<input type='url'>
search	Search field	<input type='search'>
checkbox	Checkbox for true/false options	<input type='checkbox'>
radio	Radio button for selection	<input type='radio'>
file	File upload control	<input type='file'>
date	Date picker	<input type='date'>
datetime-local	Local date and time	<input type='datetime-local'>
month	Month picker	<input type='month'>
week	Week picker	<input type='week'>
time	Time picker	<input type='time'>
range	Slider control	<input type='range'>
color	Color picker	<input type='color'>
submit	Submit button	<input type='submit'>
reset	Reset form	<input type='reset'>
button	Generic button	<input type='button'>
hidden	Hidden input field	<input type='hidden'>

Introduction to CSS

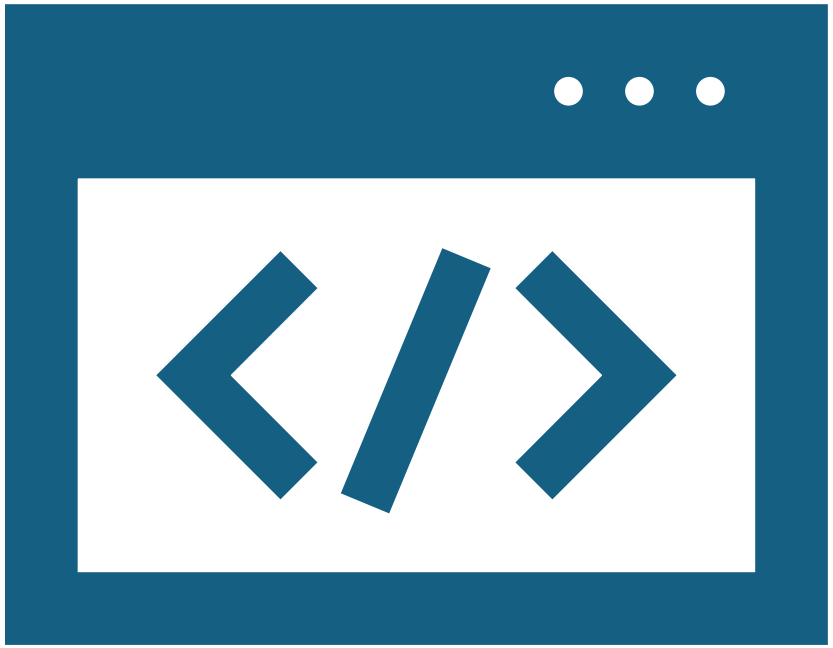
What is CSS?

- **CSS** stands for **Cascading Style Sheets**
- It is a stylesheet language for describing the appearance and formatting of HTML documents
- CSS controls layout, colors, fonts, spacing, and more
- It separates content (HTML) from presentation (CSS)

Why Use CSS?

- Keeps HTML code focused on structure
- Makes websites visually attractive and professional
- Consistent design across multiple pages
- Easy to make site-wide changes: change one CSS file and all linked pages update
- Faster development and easier maintenance

How to add CSS to HTML



There are **3** ways in which CSS can be added to a HTML file

1. **Inline CSS**

- Add style directly to an element using the **style** attribute
- Example: This is blue text.

2. **Internal CSS**

- Add styles inside a **<style>** tag within the **<head>**
- Example:

```
<style>
        p {color: blue; }
</style>
```

3. **External CSS**

- Link to a separate **.css** file
- Example: <link rel="stylesheet" href="styles.css">

CSS Syntax and Selectors

```
selector {
        property: value;
}
```

- Selectors choose which HTML elements to style:
- p targets all paragraphs
- .myClass targets all elements with class "myClass"
- #myId targets the element with id "myId"
- div > p targets <p> elements that are direct children of

Common CSS Properties

Category	Property	Description
Box Model	margin	Space outside the element's border
	padding	Space inside the element, between content and border
	border	Defines border thickness, style, and color
	width / height	Specifies dimensions of an element
	box-sizing	Includes padding and border in total width and height (border-box)
Positioning	position	`static`, `relative`, `absolute`, `fixed`, `sticky`
	top, right, bottom, left	Offsets element (used with `position`)
	z-index	Layer stacking order
Display Types	display	`block`, `inline`, `inline-block`, `flex`, `grid`, `none`
	visibility	Show/hide element (`visible`, `hidden`)
Flexbox	display: flex	Enables flex layout
	flex-direction	`row`, `column`, `row-reverse`, `column-reverse`
	justify-content	Horizontal alignment options
	align-items	Vertical alignment options
	flex-wrap	`wrap`, `nowrap`
	gap	Spacing between children
	flex-grow / shrink / basis	Flex item expansion and shrink behavior

Common CSS Properties

Grid	display: grid	Enables grid layout
	grid-template-columns / rows	Defines layout of columns/rows
	grid-column / grid-row	Span across columns/rows
	gap	Spacing between grid cells
	place-items	Shorthand for align-items and justify-items
Typography	font-size	Sets font size
	font-family	Defines typeface(s)
	font-weight	`normal`, `bold`, etc.
	line-height	Spacing between lines of text
	text-align	`left`, `right`, `center`, `justify`
	color	Text color
	text-decoration	`none`, `underline`, `line-through`
Backgrounds	background-color	Sets background color
	background-image	Sets image background
	background-size	`cover`, `contain`
Borders & Radius	border	All-in-one shorthand
	border-radius	Rounds element corners
Shadows & Effects	box-shadow	Applies shadow to elements
	text-shadow	Applies shadow to text

Flexbox Layout

- Flexbox makes it easy to design flexible, responsive layouts
- Parent uses display: flex;
- Properties:
  - flex-direction: Row or column
  - justify-content: How to align children horizontally
  - align-items: How to align children vertically
- Example:

```
.container {
 display: flex;
 flex-direction: row;
 justify-content: space-between;
align-items: center;  }
```

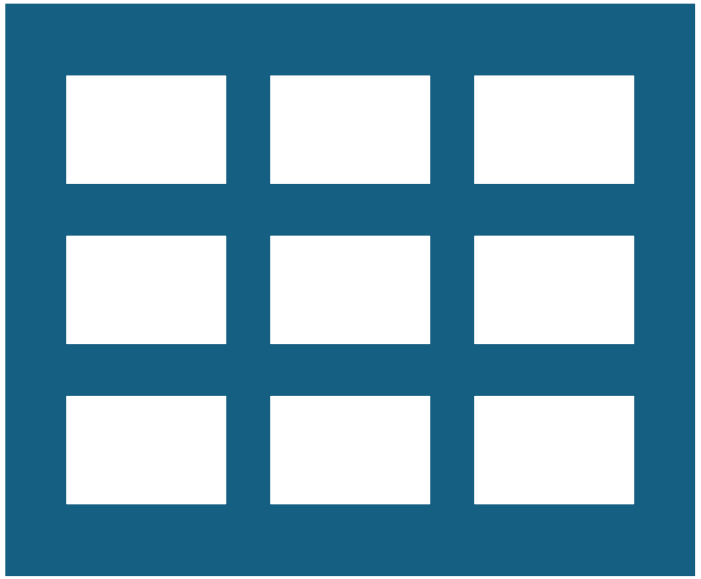
```
<div class="container">
      <div>Item 1</div>
       <div>Item 2</div>
      <div>Item 3</div>
</div>
```

Grid Layouts

- Grid is a powerful system for 2-dimensional layouts (rows and columns)
- Parent uses display: grid;
- Define columns and rows with grid-template-columns and grid-template-rows
- Example:

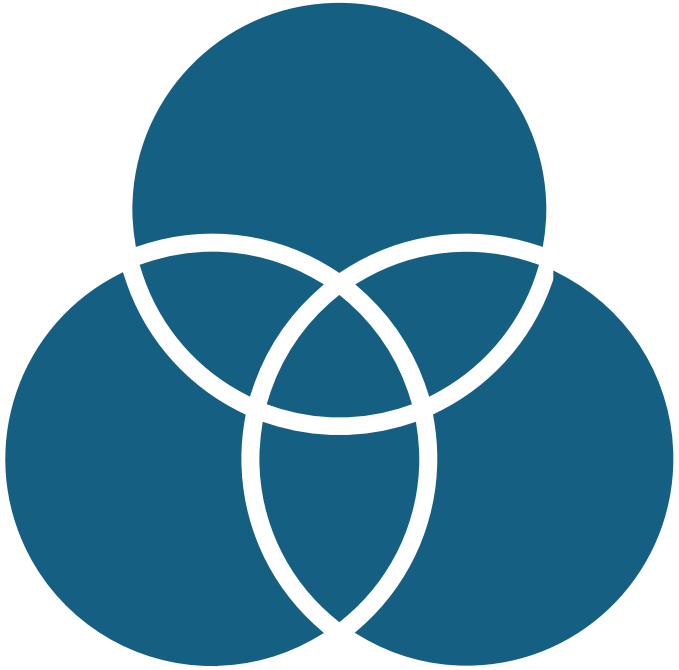
```
.grid-container {
        display: grid;
        grid-template-columns: 1fr 2fr;
        gap: 10px;
}
```

```
<div class="grid-container">
    <div>Item1</div>
    <div>Item 2</div>
</div>
```



- Grids are great for page layouts, image galleries, and more

Working with Classes and IDs



- Classes: Used for groups of elements

HTML: This is a note.

CSS: note { color: orange; }

- IDs: Used for unique elements

HTML: <h2 id="main-heading">Welcome</h2>

CSS: #main-heading { text-align: center; }

Combining Selectors and Grouping

- Grouping selectors: Apply the same style to multiple elements

```
h1, h2, h3 { color: navy; }
```

- Descendant selectors: Style elements inside other elements

```
div p { color: green; }
```

- Pseudo-classes: Style elements based on their state

```
a:hover { color: red; }
```

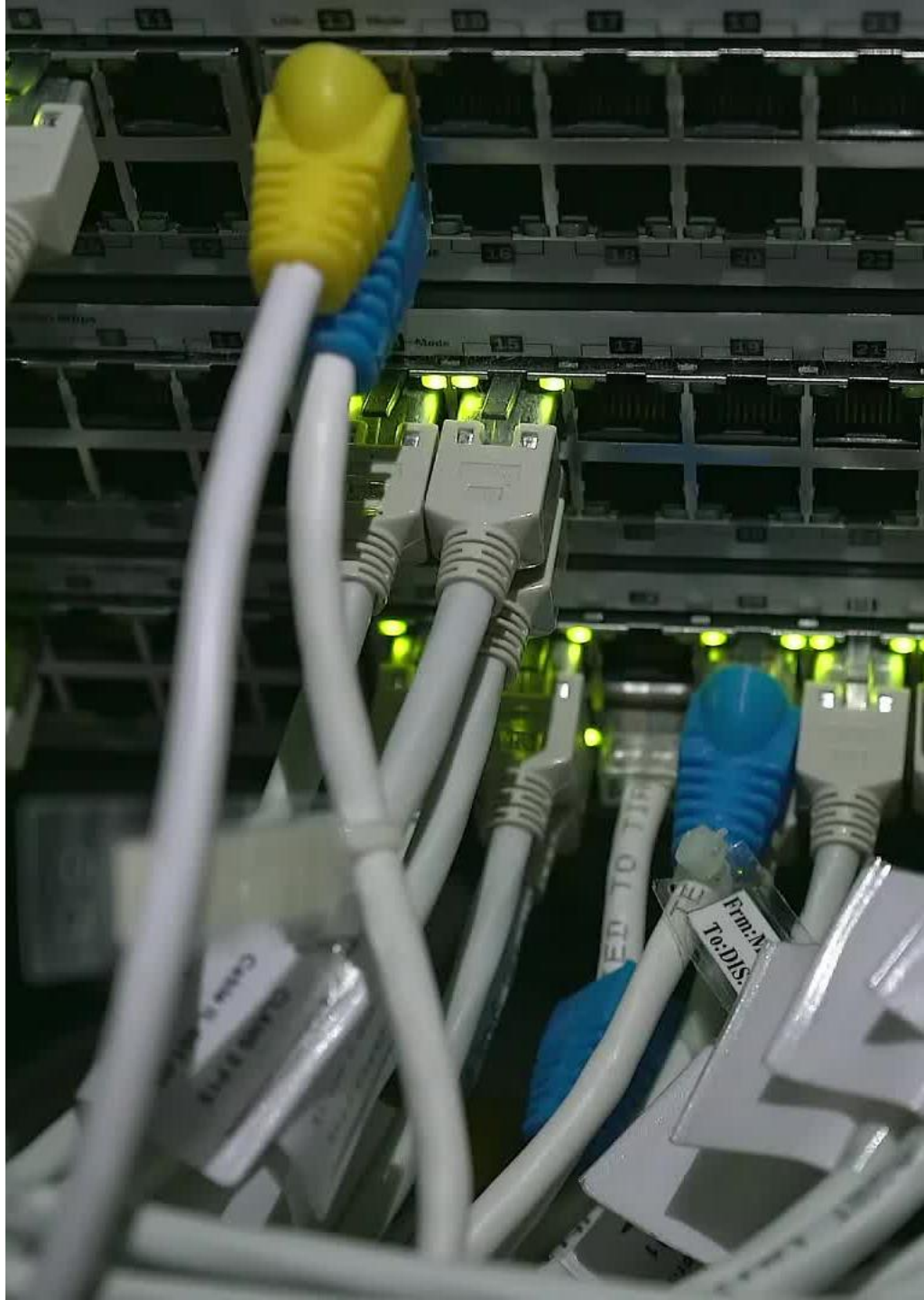
CDN – Content Delivery Network

What is a CDN?

Content Delivery Network (CDN)

A **CDN** is a network of servers distributed across the world.

- It delivers web content (like CSS, JavaScript, images) quickly to users, no matter where they are.
- Using a CDN:
  - Reduces load time by serving files from the nearest server
  - Increases reliability and availability
  - Reduces bandwidth costs
  - Essential for including popular libraries like Bootstrap, jQuery, etc.



Bootstrap

What is Bootstrap?

Bootstrap is a **free**, **open-source** front-end framework originally developed by **Twitter**. It is widely used for building responsive, mobile-first, and visually consistent websites and web applications. It allows developers and designers to create modern UIs with **minimal custom CSS or JavaScript**.

Feature	Details
Responsive Design	Uses a flexible grid system and utility classes to ensure websites look great on all devices.
Mobile-First Approach	Prioritizes mobile layout and scales up for larger devices.
Pre-Built Components	Includes UI elements like buttons, navbars, modals, forms, cards, and alerts.
Customizable	Supports easy theming with Sass variables and configuration tools.
Cross-Browser Compatible	Works consistently across major browsers.
Grid System	12-column layout system with nesting, alignment, and responsive breakpoints.
Utility Classes	Includes spacing, color, display, shadow, and positioning helpers.
JavaScript Plugins	Interactive elements like modals and carousels using JS (or jQuery in older versions).
Community & Docs	Extensive documentation, tutorials, themes, and large developer community.

Adding Bootstrap Using CDN

- The Easiest Way to Use Bootstrap
- No need to download or install anything.
- Just add one line in the <head> of your HTML:

<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">

- For JavaScript features (modals, dropdowns, etc.):

<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>

- Your page will instantly have Bootstrap's styles and functionality.

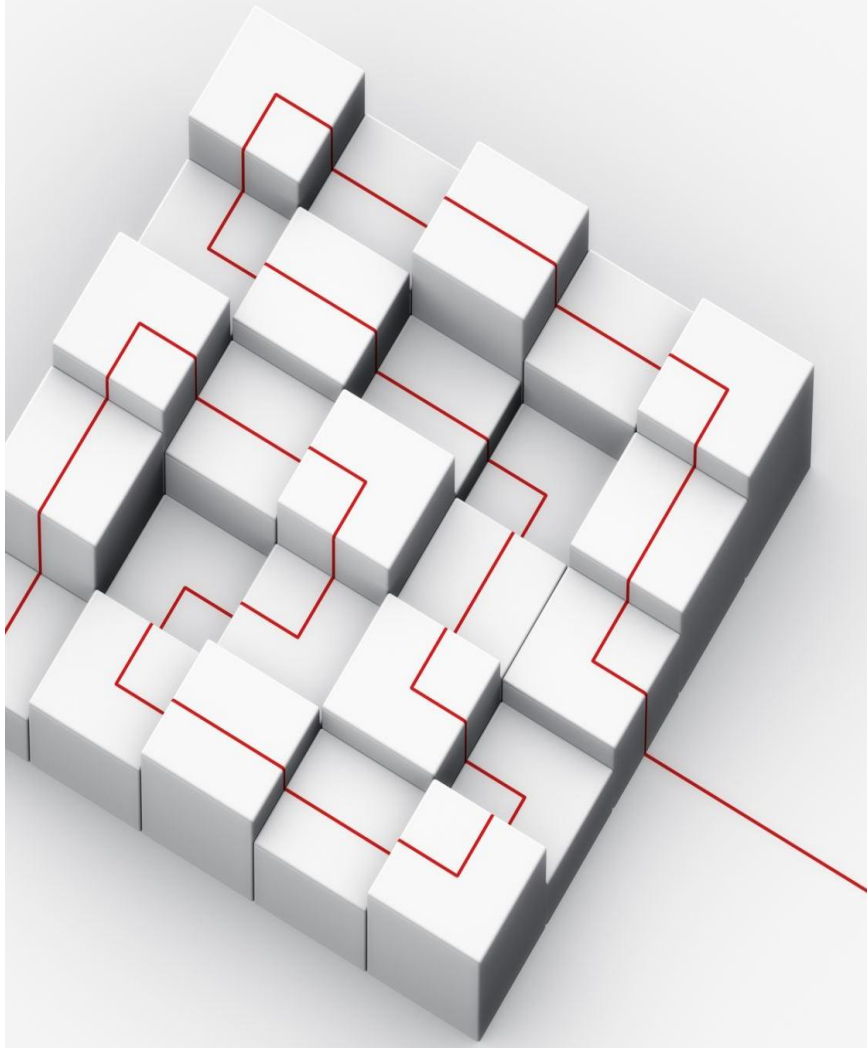
Bootstrap Grid System

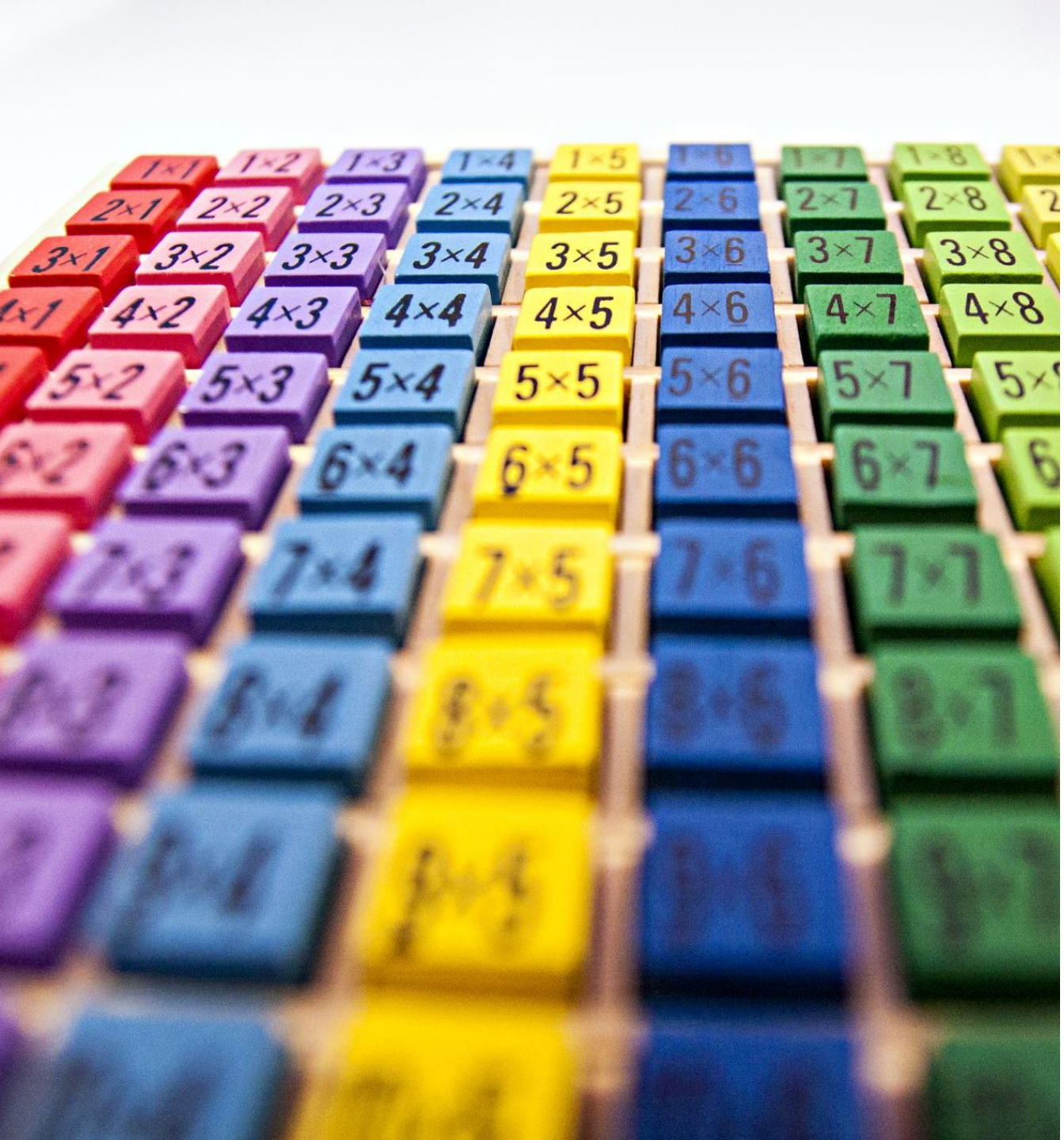
Creating Responsive Layouts Easily

- Bootstrap uses a **12-column grid system**.
- This lets you create responsive layouts that adjust to different screen sizes.
- Example:

```
<div class="container">
        <div class="row">
                <div class="col-md-4">Column 1</div>
                <div class="col-md-4">Column 2</div>
                <div class="col-md-4">Column 3</div>
         </div>
</div>
```

- **col-md-4** means 3 equal columns on medium and large screens.





Bootstrap Components

Ready-to-Use UI Elements

Bootstrap provides many reusable components:

- **Buttons**: <button class="btn btn-primary">Click Me</button>
- **Alerts**: Success!
- **Navbar**: Easy navigation bars for your site
- **Cards, modals, forms, carousels, and more**
- Components are customizable and responsive by default.

Customizing Bootstrap

Making Bootstrap Match Your Style

You can override Bootstrap's default styles with your own CSS.

```
.btn-primary {
        background-color: #4CAF50;
        border-color: #388E3C;
}
```

- Add your custom CSS after the Bootstrap CDN link.
- You can also use **Bootstrap Themes** for different looks.
- Always test your changes to ensure your site still looks good on all devices.

Introduction to JavaScript

What is JavaScript?

- JavaScript is a **programming language** used to make web pages interactive.
- Runs in the browser (client-side) and can also run on servers (Node.js).
- Common uses: interactive forms, animations, games, dynamic content updates.

Why Learn JavaScript?

- Essential for modern web development (along with HTML & CSS).
- Allows you to respond to user actions (clicks, input, etc.).
- Used by all major websites and web applications.
- Huge job market and active developer community.

How to Add JavaScript to Your Webpage

1. **Inline JavaScript**

JavaScript is written directly inside the HTML tag using the onclick or other event attributes.

<button onclick="alert('Hello!')">Click Me</button>

2. **Internal Script**

JavaScript is placed inside a <script> tag within the HTML document, usually inside the <head> or just before </body>

<script>

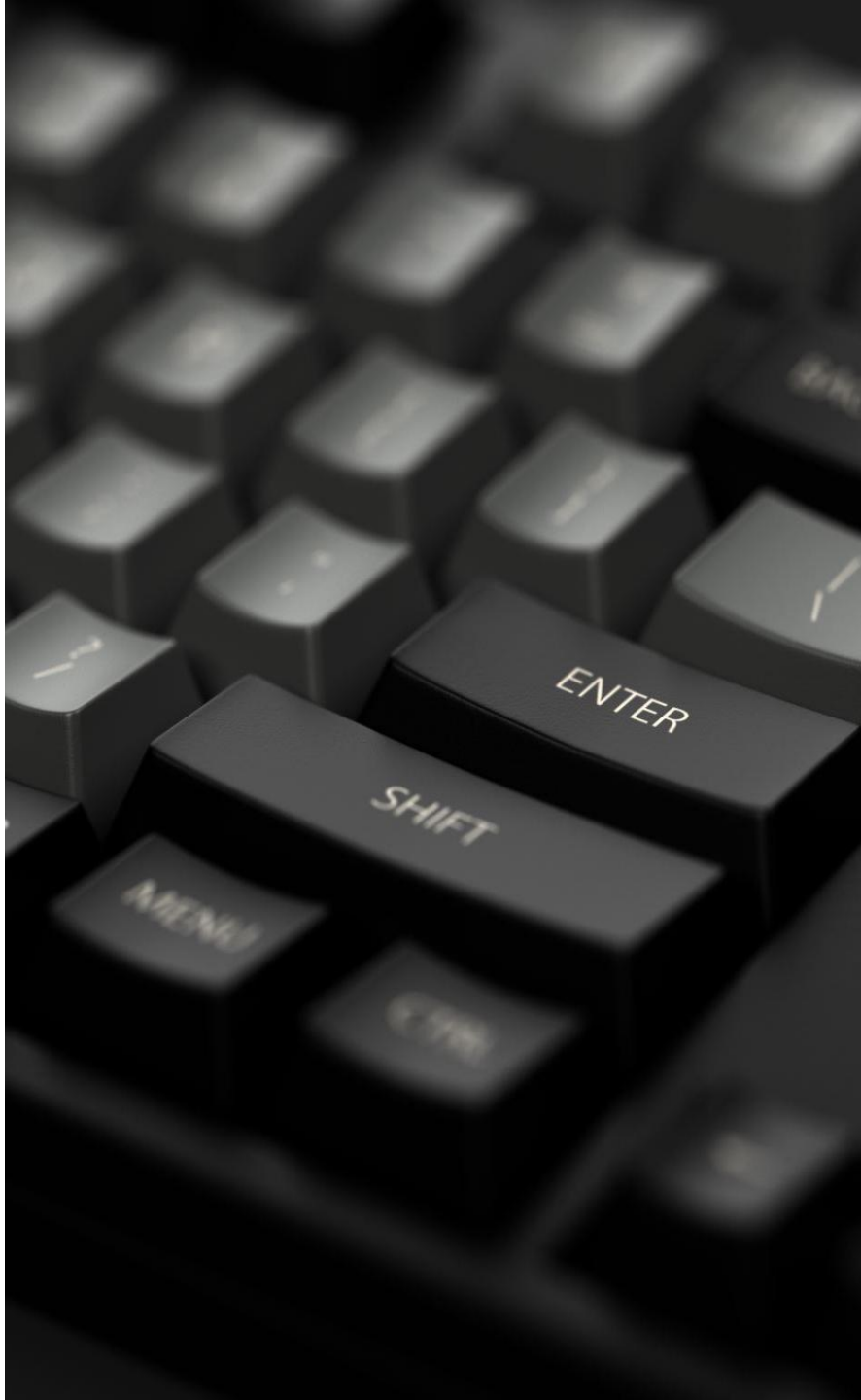
  alert('Welcome to JavaScript!');

</script>

3. **External Script**

JavaScript is written in a separate .js file and linked using the src attribute in the <script> tag

<script src="script.js"></script>



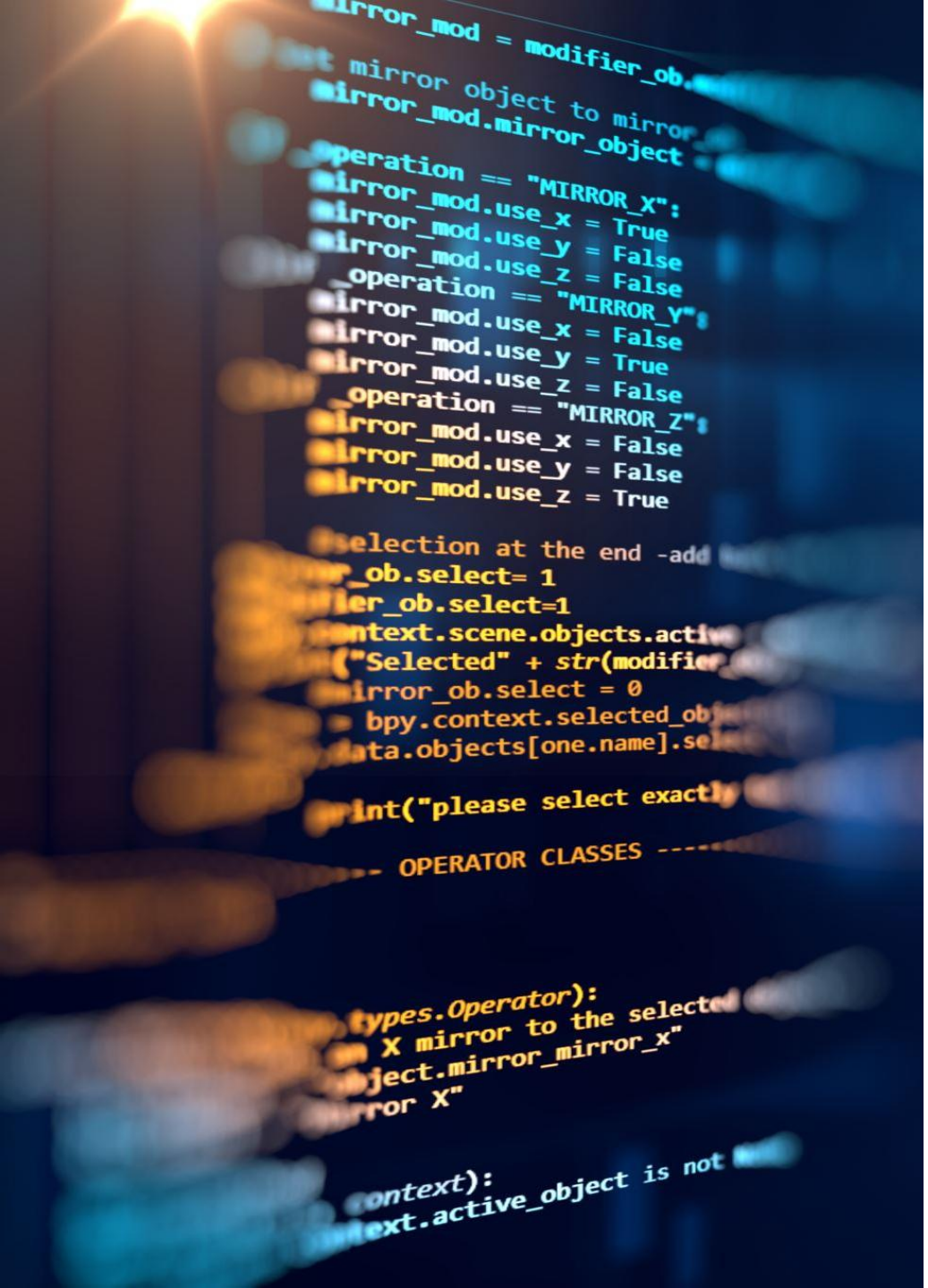
JavaScript Syntax Basics

Statements end with a semicolon (`;`)

Comments

- Single-line: // This is a comment
- Multi-line: /* This is a multi-line comment */

Case sensitive: myVar is different from **myvar**



Variables in JavaScript

- Used to store data (numbers, text, etc.)
- Declaration keywords:
  - var (Old, avoid using)
    - ❖Function-scoped, not limited to block {}
    - ❖Can be redeclared and reassigned in the same scope.
    - ❖Hoisted to the top of its scope (declaration only).
    - ❖Can lead to bugs due to unexpected scope behaviour.
  - let (Modern, block – scoped)
    - ❖Block-scoped: confined to {} where declared.
    - ❖Can be updated but not redeclared in the same scope.
    - ❖Not hoisted like var (temporal dead zone).
    - ❖Preferred for variables that change over time.
  - const (Constants)
    - Block-scoped, like let
    - Must be assigned a value at declaration.
    - Cannot be reassigned after declaration.
    - Internal properties of objects/arrays can still change.

Data Types in JavaScript

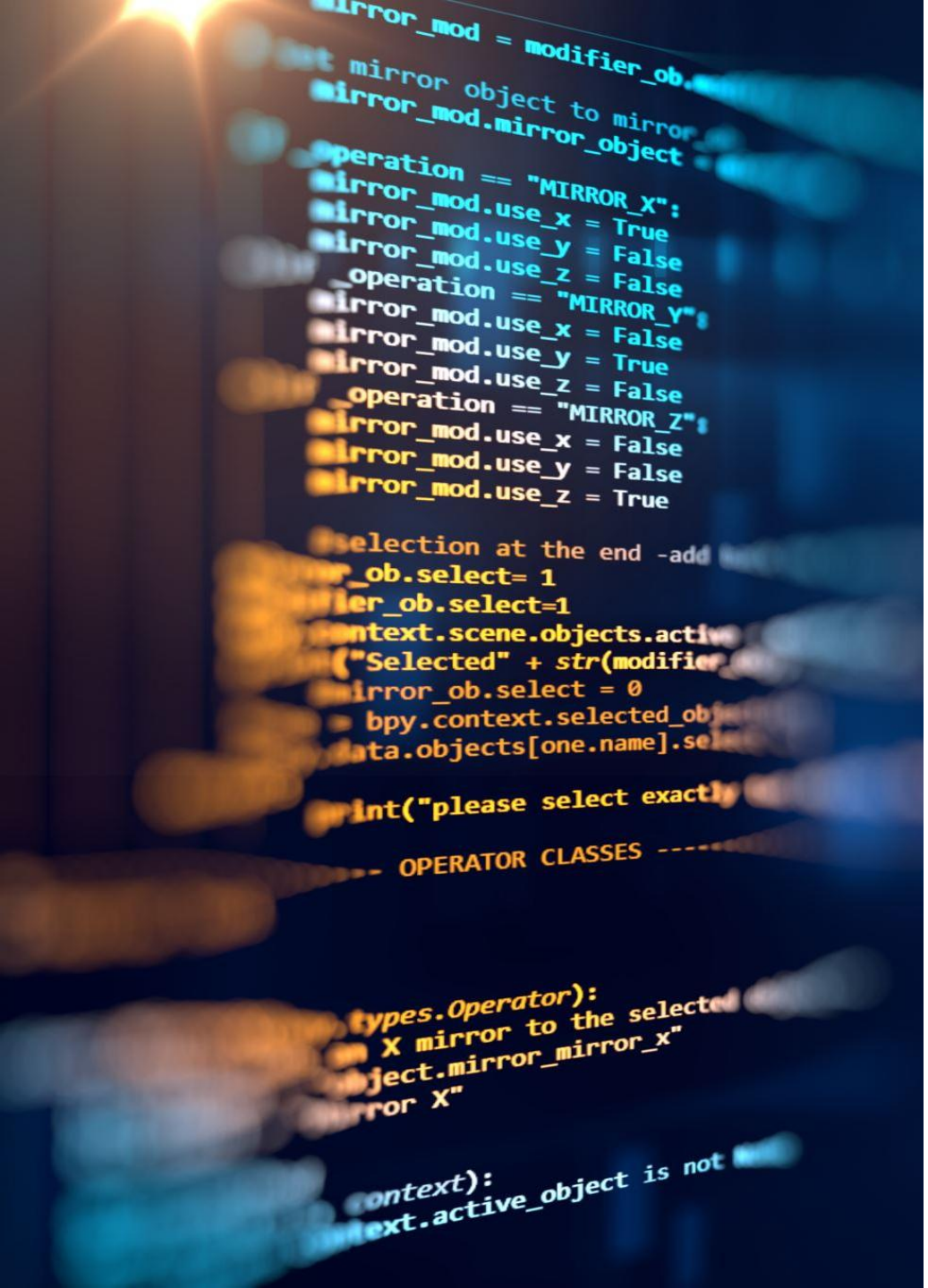
- JavaScript provides different data types to hold various kinds of values. Below is a table describing the primary data types, along with a short description and example syntax.

Data Type	Description	Syntax Example
Number	Represents numeric values (integers or floating-point).	let age = 25;
String	Sequence of characters enclosed in quotes.	let name = "Shreesha";
Boolean	Logical entity; true or false.	let isActive = true;
Undefined	Variable declared but not assigned a value.	let x;
Null	Represents intentional absence of value.	let data = null;
Object	Collection of key-value pairs.	let person = { name: "Shree", age: 20 };
Array	Ordered list of values.	let colors = ['red', 'green', 'blue'];
Function	Reusable block of code.	function greet() { alert('Hi'); }
Symbol	Unique and immutable primitive value.	let sym = Symbol('id');
BigInt	Used for large integers beyond Number limits.	let big = 1234567890123456789012345678 90n;

Operators in JavaScript

JavaScript includes various operators used to perform different kinds of operations. Below is a table listing the common operator types, a brief description, and example syntax.

Operator Type	Description	Syntax Example
Arithmetic	Used for mathematical operations like addition, subtraction, etc.	let sum = a + b;
Assignment	Assigns values to variables.	let x = 10;
Comparison	Compares two values and returns a Boolean.	a === b
Logical	Performs logical operations and returns a Boolean.	a && b
Unary	Operates on a single operand.	x++ or typeof x
Ternary	Short-hand for if-else statements.	let result = (a > b) ? 'Yes' : 'No';
Bitwise	Operates on binary representations of numbers.	a & b
String	Used to concatenate strings.	let name = 'Hello' + ' World';
Typeof	Returns the data type of a variable.	typeof x
Delete	Removes a property from an object.	delete obj.key;



Functions in JavaScript

Functions are a fundamental part of JavaScript and are used to organize code into reusable blocks.

What is a Function?

- A function is a named section of code that performs a specific task.
- Think of it as a “mini-program” inside your main program.
- You can define a function once and use it as many times as you need without rewriting the code.

Why Use Functions?

- **Reusability:** Write a block of code once and use it anywhere in your program.
- **Organization:** Makes large programs easier to understand and manage by dividing them into smaller pieces.
- **Maintainability:** If you need to change the logic, you only have to update it in one place.
- **Avoids Repetition:** Functions help avoid copying and pasting the same code multiple times.

Functions in JavaScript

How Functions Work:

1. **Defining a Function:** You give your function a name and specify what it should do.
2. **Calling a Function:** You use the function's name to "run" or "execute" the code inside it.
3. **Parameters:** Functions can take in information (inputs) called parameters, allowing them to work with different data each time they're called.
4. **Return Value:** A function can send back a result to the part of the program that called it, so you can use the output elsewhere.

Types of Functions:

Named Functions: Have a specific name and can be reused.

Anonymous Functions: Don't have a name and are often used temporarily.

Arrow Functions: A shorter way to write functions in modern JavaScript.

Conditional Statements in JavaScript

Conditional statements are a way for your JavaScript program to make decisions and respond differently depending on certain conditions or user input.

What are Conditionals?

They allow your code to ask questions and take different actions based on the answers.

For example: “If the user is logged in, show their profile; otherwise, show the login button.”

Key Concepts:

if statement: The most basic form. It checks if a condition is true. If yes, it runs a block of code.

else statement: Used with if. It runs a different block of code if the condition is false.

else if statement: Checks additional conditions if the first one is false.

Conditional Statements in JavaScript

How it Works:

- The program evaluates a condition (like “Is the age greater than 18?”).
- If it’s true, one set of instructions runs.
- If it’s not true, another set can run.

Why Are Conditionals Important?

- They make your websites and applications interactive and dynamic.
- Without them, your program would always do the same thing, no matter what the user does.

Real-Life Example:

- Imagine an ATM: If you enter the correct PIN, it lets you withdraw money. If not, it shows an error.
- That’s a conditional at work!

Loops in JavaScript

Loops let you repeat actions in your code multiple times, which is useful when you want to do the same thing for many items or until a certain condition is met.

What are Loops?

- Loops are instructions that tell your program to “keep doing this” until a specific condition is no longer true.
- They help you avoid writing the same code over and over.

Types of Loops:

1. **For Loop:** Used when you know ahead of time how many times you want to repeat an action (like printing numbers 1 to 10).
2. **While Loop:** Used when you want to keep repeating an action as long as a condition is true, but you might not know in advance how many times that will be.

Loops in JavaScript

How Loops Work:

- The loop starts.
- It checks a condition (like “Is there another item in the list?”).
- If the answer is yes, it runs the code inside the loop.
- It repeats this process until the condition is no longer true.

Why Are Loops Important?

- They make your code shorter, neater, and more powerful.
- Loops are essential for tasks like going through all items in a list, creating dynamic content, or repeating calculations.

Real-Life Example:

- Imagine you have a stack of letters to send. You pick one letter, put it in an envelope, and repeat this for each letter until the stack is gone.
- That’s a loop in action!

Arrays in JavaScript

Arrays are special variables in JavaScript that can store multiple values in a single place.

What is an Array?

- Think of an array as a list or collection—like a row of boxes, each holding a separate value.
- Each value in the array is called an “element.”
- Arrays can store numbers, strings, or even other arrays and objects.

How Arrays Work:

- Each element in an array has a position number, called an “index.”
- The first element is at position 0, the second at 1, and so on.
- You can access, add, or change elements by referring to their index.
- Arrays make it easy to work with lists of items, such as a list of names, scores, or products.

Arrays in JavaScript

Why Use Arrays?

- They help you organize large amounts of data efficiently.
- You can loop through arrays to process each item automatically (for example, displaying a list of products on a website).
- Arrays are fundamental for tasks like managing user input, storing results, or grouping related data.

Real-Life Example:

- Imagine a row of mailboxes, each holding a letter. The row as a whole is the array, and each mailbox (with its letter) is an element.

Objects in JavaScript

Objects allow you to group related information together as a single unit.

What is an Object?

- An object is like a container for properties, where each property has a name (key) and a value.
- Properties can store any type of data: numbers, strings, arrays, or even other objects.
- Objects are perfect for representing real-world items that have characteristics.

How Objects Work:

- Each property in an object describes something about that object.
- You access properties using their names.
- Objects are flexible—you can add, change, or remove properties at any time.

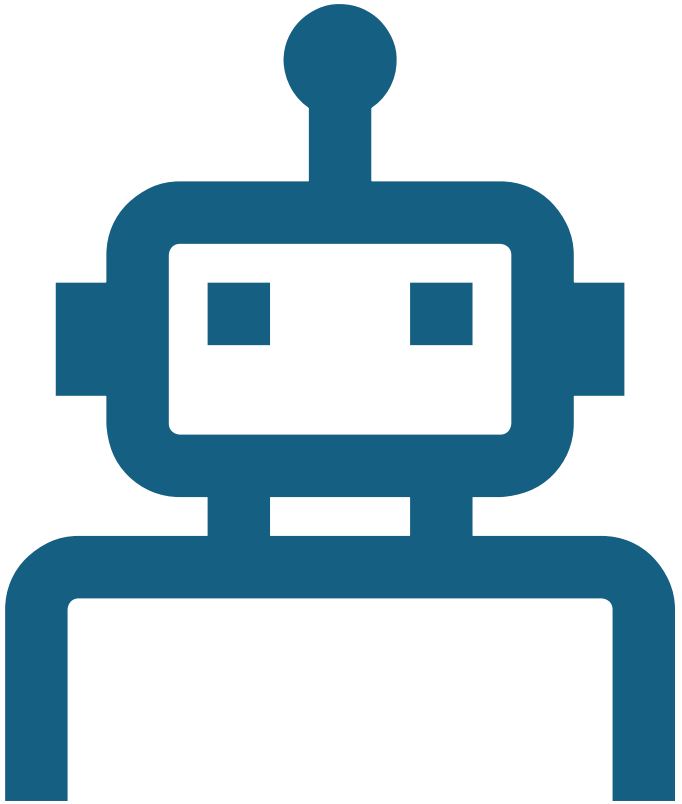
Objects in JavaScript

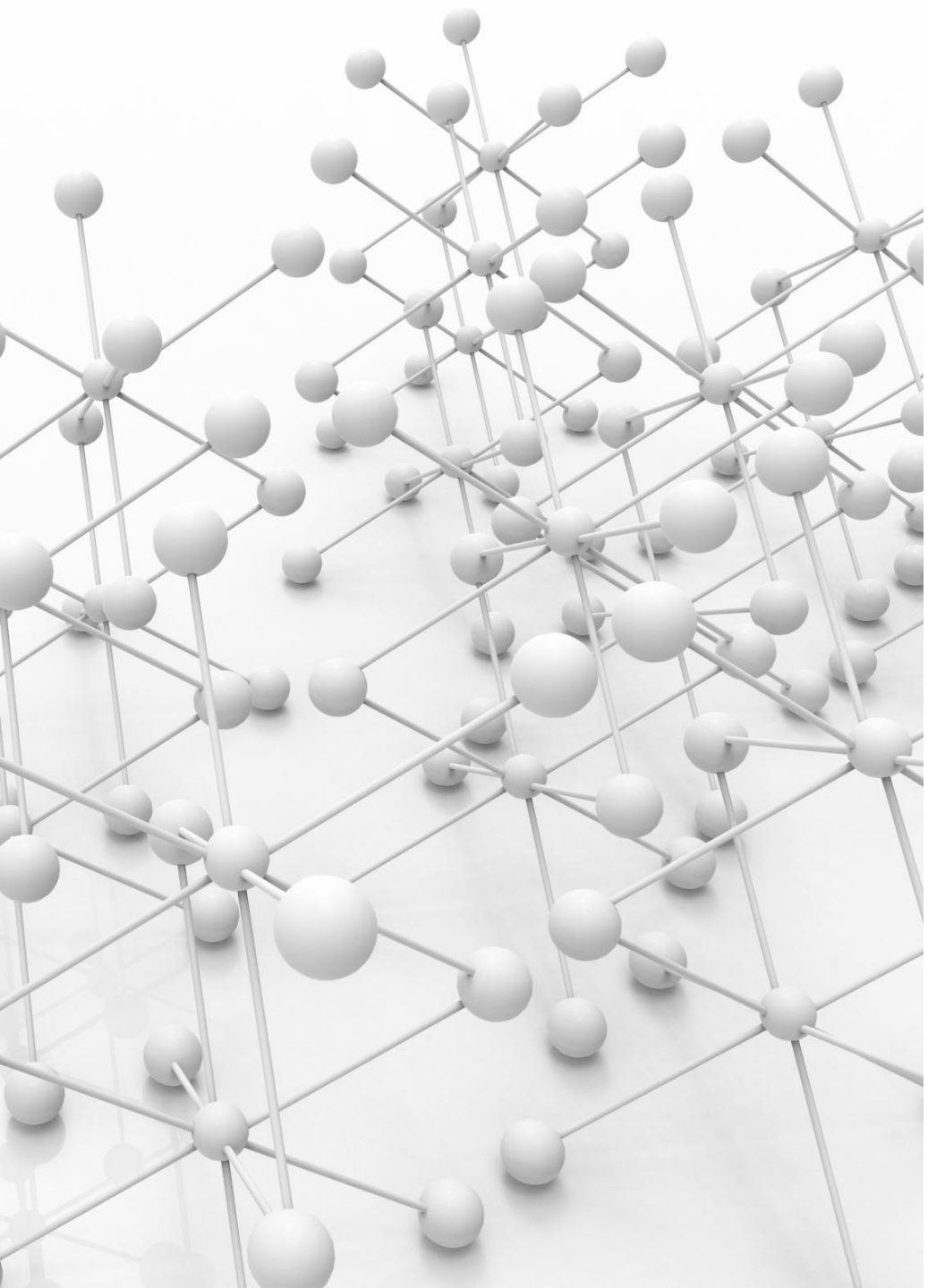
Why Use Objects?

- They’re great for organizing complex data, especially when items have many attributes.
- Objects help you model things like users, products, cars, or anything with multiple characteristics.
- They make your code clearer and more manageable by grouping related data together.

Real-Life Example:

- Think of a contact in your phone. The contact’s name, phone number, and address are all properties of a single contact object.





Interacting with the Browser (DOM)

JavaScript can change what users see on a web page by interacting with the DOM (Document Object Model).

What is the DOM?

- The DOM is a virtual tree-like structure that represents all the content and elements on a web page.
- Every element (like headings, paragraphs, buttons) is a “node” in this tree.

How Does JavaScript Use the DOM?

- JavaScript can access, change, add, or remove elements and content on the page, even after the page has loaded.
- For example, it can change text, show or hide images, create new buttons, or update styles in response to user actions.

Interacting with the Browser (DOM)

Command	Description
document.getElementById(id)	Returns the element with the specified ID.
document.getElementsByClassName(className)	Returns a live HTMLCollection of all elements with the specified class name.
document.getElementsByTagName(tagName)	Returns a live HTMLCollection of elements with the given tag name.
document.querySelector(selector)	Returns the first element that matches a specified CSS selector(s).
document.querySelectorAll(selector)	Returns a static NodeList representing a list of elements that match the specified group of selectors.
element.innerHTML	Gets or sets the HTML or XML markup contained within the element.
element.textContent	Gets or sets the text content of the specified node.
element.setAttribute(name, value)	Sets the value of an attribute on the specified element.
element.getAttribute(name)	Returns the value of a specified attribute on the element.
element.removeAttribute(name)	Removes the specified attribute from the element.
document.createElement(tagName)	Creates the HTML element specified by tagName.
parentNode.appendChild(node)	Adds a node to the end of the list of children of a specified parent node.
parentNode.insertBefore(newNode, referenceNode)	Inserts a node before a reference node as a child of a specified parent node.
node.removeChild(child)	Removes a child node from the DOM and returns the removed node.
element.addEventListener(event, function)	Attaches an event handler to the specified element.
element.removeEventListener(event, function)	Removes an event handler that was previously attached.
element.classList.add(className)	Adds the specified class value.
element.classList.remove(className)	Removes the specified class value.
element.classList.toggle(className)	Toggles the existence of a class in the element's class list.
element.style.property	Gets or sets the style properties of an element.

JavaScript Events

What are Events?

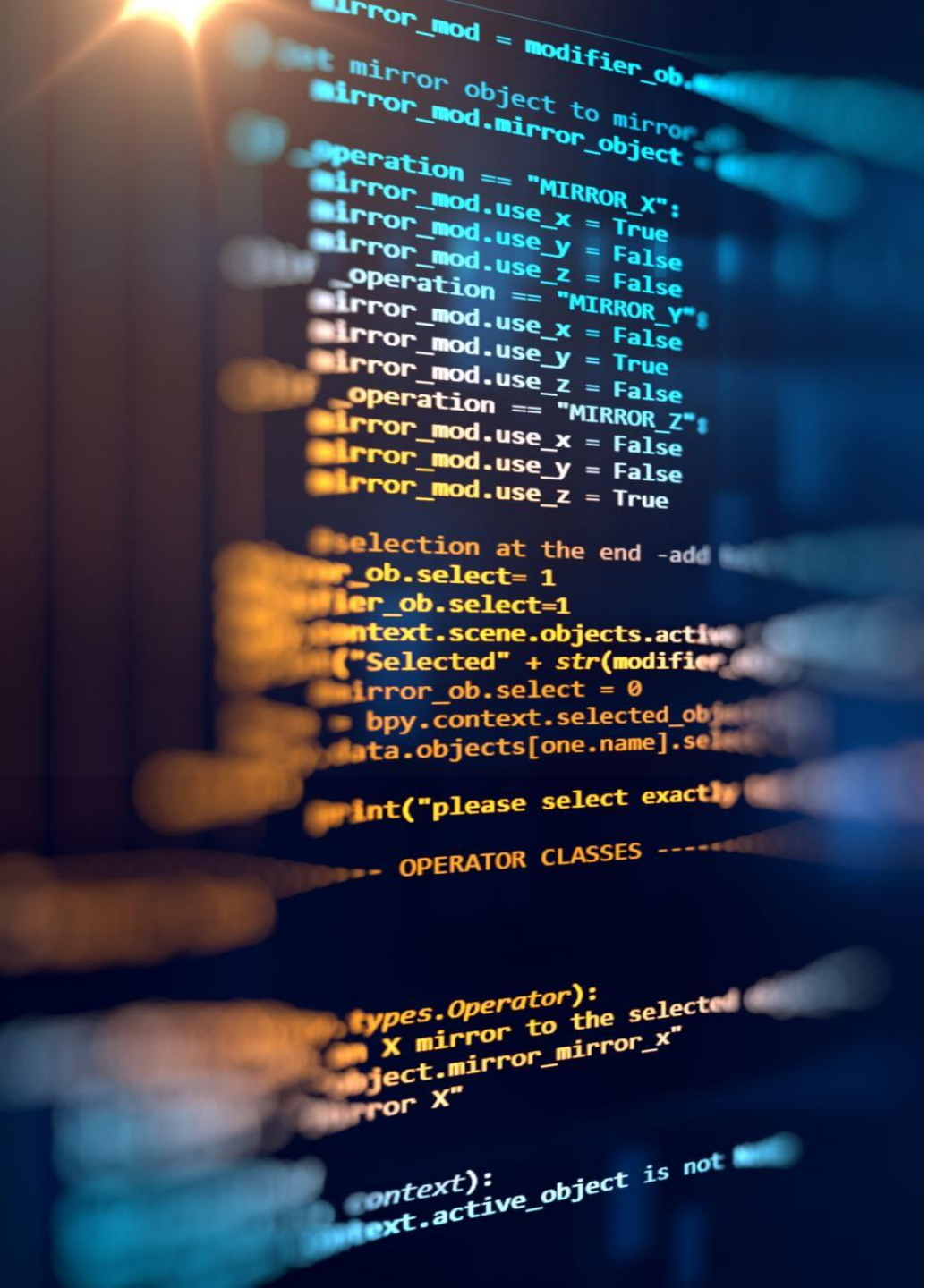
- Events are actions or occurrences that happen in the browser, which the browser can detect and respond to.
- Examples include a user clicking a button, moving the mouse, typing in a text box, loading a page, or resizing the browser window.

How Events Work:

- **Event Occurs:**
  - ❖ Something happens on the page (like a mouse click or key press).
- **Event Listener:**
  - ❖ JavaScript can "listen" for these events by setting up event listeners.
- **Response:**
  - ❖ When the event happens, the browser notifies your JavaScript code, which can then run specific instructions (called "event handlers").

Why are Events Important?

- They allow your web page to react to user actions, making it interactive and dynamic.
- Without events, web pages would be static and could not respond to what users do.



Forms and How They're Processed: Client-Side v/s Server-Side

What is a Form?

- A form is a section of a web page where users can input data, such as text, numbers, or selections, and submit it for processing.
- Examples: login forms, contact forms, search bars, registration forms.

Client-Side Processing

- Happens in the user's browser, using JavaScript.
- Data validation (checking if fields are filled, email is valid, etc.) can be done instantly before the form is sent to the server.
- Provides quick feedback to users and reduces server load.
- Example: Showing an error message if the email field is empty, without needing to reload the page.

Server-Side Processing

- Happens on a web server, after the form is submitted.
- Handles tasks like saving data to a database, sending emails, or logging in users.
- More secure: all important checks (like passwords) must be done on the server, since client-side checks can be bypassed.

Form Control

Form controls are the interactive elements inside a form where users input data. Common types include:

Text Control: For single-line text input (e.g., name, email).

Password Control: For passwords (input is hidden).

Radio Buttons: For selecting one option from many.

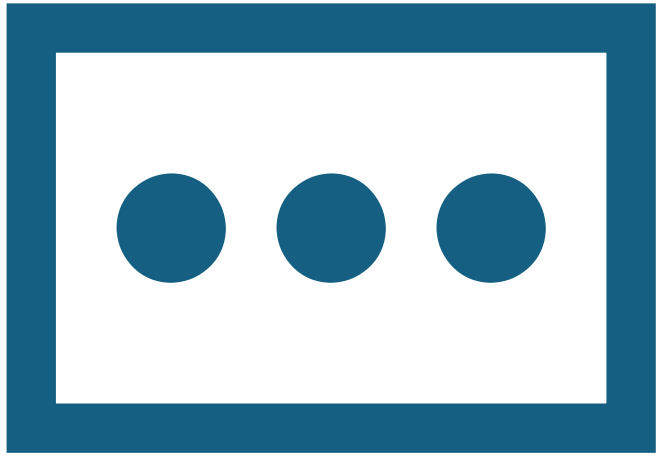
Checkboxes: For selecting multiple options.

Textarea: For multi-line text input.

Select (Dropdown): For picking one (or more) options from a list.

Buttons: For submitting or resetting the form.

Accessing a form’s control values in JavaScript

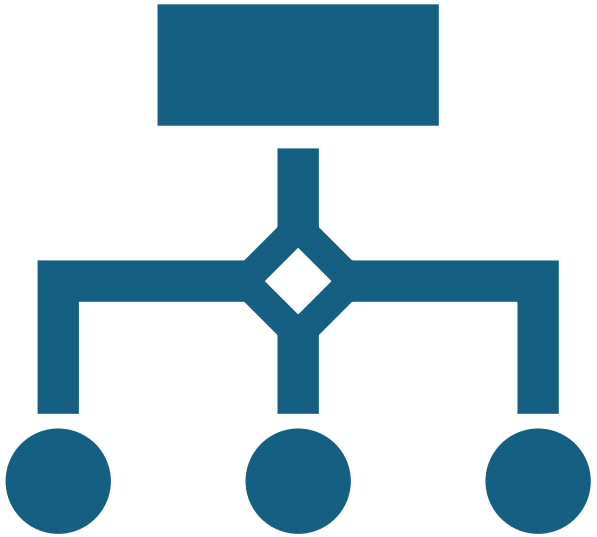


- JavaScript can read the data entered by the user before submitting the form.
- This allows you to validate input, show messages, or use the data elsewhere.

How it’s done:

- Every form and its controls can be accessed using the DOM.
- Each control can be referred to by its name or id
- Example scenario: When a user clicks “Submit”, JavaScript grabs the values the user entered and checks if they’re valid.

Reset and Focus Methods



reset() Method

- Belongs to the form element.
- When called, it clears all the controls in the form, returning them to their default values (as they were when the page loaded).
- Useful for “Reset” buttons, letting users quickly clear their input.

focus() Method

- Belongs to form controls (like input fields).
- Moves the cursor to a specific field, so the user can start typing immediately.
- Useful for improving usability (e.g., automatically placing the cursor in the first field when the page loads, or after an error).